

جامعة الأقصى
كلية الحاسبات وتكنولوجيا المعلومات
قسم الهندسة والشبكات



جامعة الأقصى
كلية الحاسبات وتكنولوجيا المعلومات
قسم الهندسة والشبكات

أنظمة التشغيل للأنظمة الذكية (ENG2306)

المحاضرة: 6 أساسيات برمجة Bash

المحاضر: المهندس محمد الهمص

المحاضرة 6: أساسيات برمجة Bash

أهداف اليوم:

- 1. فهم بنية البرنامج النصي الأساسي لـ Bash.
- 2. تعلم كيفية استخدام المتغيرات لتخزين واسترجاع البيانات.
- 3. تنفيذ عملية اتخاذ القرار باستخدام عبارات if-then-else.
- 4. أتمتة المهام المتكررة باستخدام حلقات for و while.
- 5. اكتب نصًا بسيطًا لأتمتة مهمة النظام.

ملخص: لماذا نحتاج إلى البرمجة النصية؟

خلال الأسابيع الأربعة الماضية، تعلمنا العديد من الأوامر القوية:

ls, mkdir, chmod, chown (إدارة الملفات)

useradd, usermod, groupadd (إدارة المستخدمين)

ps, kill, nice (إدارة العمليات)

المشكلة: إن كتابة هذه الأوامر يدويًا واحدًا تلو الآخر أمر بطيء ومتكرر وعرضة للأخطاء المطبعية.

الحل: الأتمتة!

البرنامج النصي هو ملف نصي بسيط يحتوي على سلسلة من الأوامر. يُنفَّذ shell هذه الأوامر بالترتيب، محولاً مهمة يدوية متعددة الخطوات إلى إجراء واحد متكرر. هذا هو أساس الأتمتة في لينكس.

تشرح نص Bash

يعتبر نص Bash عبارة عن ملف نصي عادي يحتوي على ثلاث ميزات رئيسية:

1. The Shebang: #!/bin/bash

• يجب أن يكون هذا هو السطر الأول من النص الخاص بك.

• يخبر نظام التشغيل باستخدام مترجم Bash لتشغيل الأوامر الموجودة في هذا الملف.

2. التعليقات: # هذا تعليق.

• يتم تجاهل الأسطر التي تبدأ بـ # بواسطة المترجم.

• استخدم التعليقات لشرح ما يفعله نصك وكيفية عمله، التعليقات الجيدة هي ضروري لصيانة الكود.

3. إذن قابل للتنفيذ:

افتراضيًا، لا يُمكن تشغيل ملف نصي جديد كبرنامج. يجب عليك جعله قابلاً للتنفيذ أولاً.

• `chmod +x my_script.sh`

لتشغيل البرنامج النصي: `./my_script.sh` (يخبر `./` shell بالبحث في الدليل الحالي).

استخدام المتغيرات لتخزين البيانات

المتغيرات هي عبارة عن حاويات تحمل معلومات.

إعلان المتغير:
`EMAN_ELBAIRAV="بعض القيم"`

• الاتفاقية: استخدم الأحرف الكبيرة للمتغيرات البيئية والأحرف الصغيرة للمتغيرات المحلية للبرنامج النصي.

• لا تضع مسافات حول علامة `=`. هذا خطأ شائع!

الوصول إلى متغير: استخدم علامة `$` قبل اسم المتغير.
`echo $VARIABLE_NAME`

• ممارسة جيدة: قم بإحاطة المتغير بعلامتي اقتباس مزدوجتين للتعامل مع المسافات بشكل صحيح: `echo "$VARIABLE_NAME"`

مثال على البرنامج النصي (`greeting.sh`):

```
#!/bin/bash # نص برمجي بسيط يستخدم متغيرًا.
```

تحية="مرحبًا بالعالم!" صدى "\$GREETING"

الحصول على الإدخال: القراءة واستبدال الأوامر

يمكن أن تكون البرامج النصية الخاصة بك تفاعلية أو تستخدم مخرجات الأوامر الأخرى.

1. قراءة إدخال المستخدم: القراءة يوقف أمر القراءة البرنامج النصي مؤقتًا وينتظر حتى يكتب المستخدم شيئًا ما.

```
#!/bin/bash read -p "الرجاء إدخال اسمك: " user_name echo "مرحبًا, $user_name! مرحبًا بك في البرمجة النصية."
```

2. استبدال الأمر: `${...}`

تتيح لك هذه الميزة القوية حفظ مخرجات الأمر في متغير.

```
#!/bin/bash # قم بتخزين التاريخ الحالي واسم المضيف في المتغيرات my_hostname=$(hostname) current_date=$(date)
```

`echo` تم إنشاء التقرير الخاص بـ `my_hostname` في `$current_date`
للأنظمة الذكية: `cpu_temp=$(cat /sys/class/thermal/thermal_zone0/temp)`

اتخاذ القرارات: إذا-إذن-إلا

تسمح العبارات الشرطية لنصك بتنفيذ إجراءات مختلفة استنادًا إلى شرط معين.

بناء الجملة:

```
إذا [ الشرط ]; إذن
#كتلة من التعليمات البرمجية التي سيتم تشغيلها إذا كان الشرط صحيحًا
elif [ another_condition ]; then #
كتلة من التعليمات البرمجية التي سيتم تشغيلها إذا كانت جميع الشروط خاطئة
else #
كتلة من التعليمات البرمجية التي سيتم تشغيلها إذا كان الشرط صحيحًا
fi
```

يكون

•التباعد أمرٌ بالغ الأهمية! يجب أن تكون هناك مسافات بين الأقواس. [...]

[] •هو اسم مستعار لأمر الاختبار.

الحالات الشائعة:

•مقارنة السلسلة: إذا ["\$USER" == "root"]

•المقارنة العددية: إذا [-gt 10 \$count -gt 10] أكبر من، < -lt أقل من، = -eq يساوي

•فحص الملف: إذا [-f "/etc/passwd"] (-f = هو ملف، -d هو دليل)

المهام المتكررة: حلقة for

يتم استخدام حلقة for لتنفيذ كتلة من التعليمات البرمجية لعدد معين من المرات، والتكرار عبر قائمة من العناصر.

بناء الجملة:

```
للعنصر في العنصر 1 العنصر 2 العنصر 3: قم بتنفيذ #الكود الذي سيتم تنفيذه لكل
عنصر "echo" معالجه "$item" تم
```

حالة الاستخدام العملي (الأنظمة الذكية): التحقق من حالة أجهزة إنترنت الأشياء المتعددة على الشبكة.

```
# /bin/bash | إرسال أمر ping إلى قائمة أجهزة إنترنت الأشياء
```

الأجهزة="192.168.1.102 192.168.1.103" 101.1.861.291

```
للجهاز في $DEVICES: قم بتنفيذ الأمر ping -c 1 $device done
```

سيقوم هذا البرنامج النصي بإرسال أمر ping إلى كل عنوان IP واحدًا تلو الآخر.

المهام المتكررة: حلقة while

تستمر حلقة while في تنفيذ كتلة من التعليمات البرمجية طالما ظل الشرط صحيحًا.

بناء الجملة:

بينما [شرط] :افعل

#الكود الذي سيتم تنفيذه أثناء تحقق الشرط حالة الاستخدام العملية (الأنظمة الذكية): برنامج نصي للمراقبة يتحقق من درجة حرارة وحدة المعالجة المركزية كل 10 ثواني.

#!/bin/bash #مراقبة درجة حرارة وحدة المعالجة المركزية إلى أجل غير مسمى

بينما صحيح: قم بـ temp=\$(cat /sys/class/thermal/thermal_zone0/temp)

#القيمة بالملي درجة، لذا قسّم على 1000 echo "درجة حرارة وحدة المعالجة المركزية الحالية هي: \$(temp / 1000))°C sleep 10 done

هذه حلقة لا نهائية؛ يمكنك إيقافها باستخدام Ctrl+C.

سيناريو عملي: النسخ الاحتياطي البسيط للنظام نص

دعنا نكتب نصًا برمجيًا يقوم بعمل نسخة احتياطية لملف سجل معين.

1. قم بإنشاء ملف نصي جديد: touch log_backup.sh

2. اجعله قابلاً للتنفيذ: chmod +x log_backup.sh

3. افتحه وأضف الملف: #!/bin/bash

4. أضف تعليقات تشرح غرض النص.

5. قم بإنشاء متغير لوجهة النسخ الاحتياطي: BACKUP_DIR="/home/your_user/backups" (استبدل your_user).

6. قم بإنشاء متغير للتاريخ: DATE=\$(date +%Y-%m-%d)

7. تحقق من وجود مجلد النسخ الاحتياطي. إن لم يكن موجودًا، فأنشئه.

إذا [-d "\$BACKUP_DIR"] ! ثم mkdir -p "\$BACKUP_DIR"

8. حدد ملف المصدر الذي سيتم نسخه احتياطيًا: SOURCE_FILE="/var/log/syslog"

9. استخدم الأمر cp لنسخ الملف، مع إضافة التاريخ إلى اسم الملف. syslog-backup-\$DATE.log

cp "\$SOURCE_FILE" "\$BACKUP_DIR/"

10. أضف عبارة صدى لتأكيد نجاح عملية النسخ الاحتياطي.

echo "تم بنجاح نسخ \$SOURCE_FILE احتياطيًا إلى \$BACKUP_DIR"

الفكرة وراء المهمة

لماذا تعد كتابة النصوص البرمجية القوة العظمى لمسؤول أنظمة Linux؟

•الاتساق والموثوقية: يقوم البرنامج النصي بتنفيذ المهمة بنفس الطريقة تمامًا في كل مرة، القضاء على الخطأ البشري. عند تجهيز اثني عشر مستشعرًا متطابقًا لإنترنت الأشياء، يضمن البرنامج النصي تم تكوين جميعها بشكل مثالي.

•الكفاءة: يمكن إنجاز مهمة تستغرق 10 دقائق من قبل الإنسان للقيام بها يدويًا (مثل إنشاء مستخدم، وتعيين كلمة مرور، وإنشاء أدلة، وتعيين الأذونات) بواسطة برنامج نصي في أقل من ثانية.

•الأتمتة المجدولة: باستخدام أداة تسمى cron(والتي سنشير إليها)، يمكنك جعل تعمل البرامج النصية تلقائيًا. على سبيل المثال، شقّل برنامجًا نصيًا احتياطيًا كل ليلة الساعة الثانية صباحًا، أو شقّل برنامجًا نصيًا لفحص الحالة كل ساعة وسجل النتائج. هذا ضروري للحفاظ على الأنظمة الذكية التي يجب أن تعمل على مدار الساعة دون تدخل بشري.

المهمة 5: نص إعداد المستخدم التلقائي (12%)

الهدف: كتابة نص برمجي يقوم بأتمتة عملية إنشاء مستخدم جديد ودليل احتياطي له، بما في ذلك التحقق من الأخطاء الأساسية.

المهام:

- 1.قم بإنشاء البرنامج النصي باسم create_project_user.sh.
- 2.يجب أن يطالب البرنامج النصي المستخدم بإدخال اسم المستخدم باستخدام القراءة.
- 3.استخدم عبارة if للتحقق مما إذا كان المستخدم موجودًا بالفعل.
- تلميح: يمكنك استخدام الأمر u-id > اسم المستخدم<. سينجح هذا الأمر (الخروج الرمز :-) إذا كان المستخدم موجودًا، ويفشل (رمز خروج غير صفري) إذا لم يكن موجودًا. تحقق من رمز الخروج باستخدام [\$? -eq 0].
- 4.إذا كان المستخدم موجودًا بالفعل، فيجب أن يقوم البرنامج النصي بطباعة رسالة خطأ (على سبيل المثال، "خطأ: اسم المستخدم موجود بالفعل.") والخروج.
- 5.إذا لم يكن المستخدم موجودًا، فيجب أن يقوم البرنامج النصي بما يلي:
 - إنشاء المستخدم مع الدليل الرئيسي. (sudo useradd -m ...)
 - قم بإنشاء دليل لهم داخل / srv باسم project-<username>.
 - تغيير ملكية هذا الدليل إلى المستخدم الجديد.

• اطبع رسالة نجاح مثل "تم إنشاء اسم المستخدم ومشروعه بنجاح"
دليل.

تعليمات تسليم الواجبات

- اسم الملف: قم بتسمية ملف الإرسال الخاص بك كما يلي تمامًا: studentname-Assignment5.pdf studentnum-13012070xx-MohammedEmadAlhams-Assignment5.pdf مثال:
- موجه المحطة الطرفية: يجب أن يظهر اسمك في موجه المحطة الطرفية لجميع لقطات الشاشة.
- تنسيق الملف: يجب أن يكون إرسالكم عبارة عن ملف PDF واحد يحتوي على:
 - الكود الكامل لنص create_project_user.sh الخاص بك (نسخ ولصق كنص).
 - لقطة شاشة توضح تشغيل البرنامج النصي بنجاح لمستخدم جديد.
 - لقطة شاشة ثانية تُظهر فشل البرنامج النصي بشكل سليم عند محاولة إنشاء نفس المستخدم مرة أخرى.
- تقرير موجز (فقرة أو فقرتان) يشرح كيفية عمل منطق بيان if الخاص بك للتحقق مما إذا كان يوجد مستخدم يشرح على وجه التحديد غرض \$.
- النزاهة الأكاديمية:
- تحذير: سيتم اعتبار المشاركات المتطابقة غشًا وستؤدي إلى الحصول على درجة صفر.
- تحذير: الحلول أو التقارير التي تم إنشاؤها بواسطة الذكاء الاصطناعي ستؤدي إلى الحصول على درجة صفر.

الخطوات التالية والموارد

- تمرين هذا الاسبوع:
- عدّل النصوص من هذه المحاضرة. هل يمكنك جعل نص النسخ الاحتياطي يستخدم حلقة for للنسخ الاحتياطي؟ ملفات متعددة؟
- اكتب نصًا بسيطًا يطلب رقمًا ويخبر المستخدم إذا كان أكبر من 100.
- الأسبوع القادم: المحاضرة 7 - برمجة Bash المتقدمة
- سنجعل نصوصنا البرمجية أكثر قوة وقابلية لإعادة الاستخدام من خلال التعرف على الوظائف والتعامل معها
- قوائم البيانات مع المصفوفات، وتنفيذ معالجة أفضل للأخطاء.
- نصائح الاتصال بغزة:
- قم بتنزيل ملف PDF الخاص بهذه المحاضرة واحفظه دون اتصال بالإنترنت.

• اكتب واختبر نصك بالكامل دون اتصال بالإنترنت. كل ما عليك فعله هو الاتصال بالإنترنت لإرسال ملف PDF النهائي.
تقرير.

موارد:

• ورقة الغش الخاصة ببرمجة Bash: <https://devhints.io/bash>

• [تعليم - Shell البرنامج التعليمي التفاعلي: https://www.learnshell.org/](https://www.learnshell.org/)

• اقرأ الدليل الخاص بـ bash: man bash

• مواد الدورة على Moodle